

Grupo 84

Curso 2026/2027

Práctica 1. Heurística y Optimización

AUTORES:

NOMBRE	NIA	CORREO
Carla Martín Ruiz	522326	100522326@alumnos.uc3m.es
Lucía Ramírez Sánchez	550254	100550254@alumnos.uc3m.es

INTRODUCCIÓN

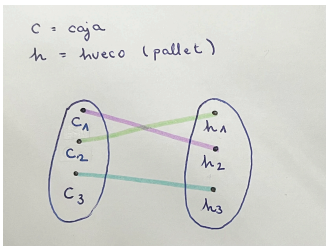
Este documento recoge la resolución de la primera práctica de Heurística y Optimización: la disposición óptima de cajas sobre pallets mediante Programación Lineal Entera, resuelta con GLPK. Para cada una de las cuatro partes se describe el modelo (tipo de problema, conjuntos y parámetros, variables de decisión, restricciones y función objetivo), su implementación en *parte-X.mod* y los casos de prueba realizados con *sol-X.py*. El CASO N de cada parte se encuentra en *parte-X/ejemplos/ejemplo-N.in*. Por último, el análisis de resultados estudia la complejidad de cada parte, su factibilidad, la existencia de varios óptimos y el tamaño máximo de los problemas que se pueden resolver.

PARTE 1. MINIMIZACIÓN DEL COSTE MEDIO

Modelización del problema:

Tipo de problema

Este primer problema corresponde a un caso de **Programación Lineal Entera**, en el que se busca determinar la disposición óptima de las cajas sobre un único pallet, minimizando el coste medio y respetando las restricciones de prioridad y acceso.



Se trata de un problema de **asignación**: cada una de las *n* cajas debe asignarse a una de las $n = l^2$ posiciones del pallet, de forma que cada caja ocupe exactamente una posición y cada posición reciba exactamente una caja. Se puede representar como un grafo bipartito en el que un conjunto de nodos son las cajas, el otro las posiciones, y cada arco tiene asociado el coste de colocar esa caja en esa posición. Es un problema **factible** y **acotado** (el número de soluciones posibles es finito).

Parámetros

l: longitud del lado del pallet.
$n = l^2$: número total de cajas y de posiciones disponibles en el pallet.
p_k: prioridad de la caja k.
K = {1,....,n}: conjunto de cajas.
F = {1,....,l}: conjunto de filas.
C = {1,....,l}: conjunto de columnas.

Variables de decisión

$x_{kij} \in \{0,\ 1\} \rightarrow$ 1 si la caja *k* se coloca en la fila *i* y la columna *j*; 0 en otro caso.

Se define una variable para cada caja y cada posición, $\forall k \in K,\ \forall i \in F,\ \forall j \in C$.

Restricciones

1. Cada caja ocupa una única posición → Para cada caja *k*, la suma de sus variables sobre todas las posiciones del pallet vale 1: $\sum_{i \in F} \sum_{j \in C} x_{kij} = 1 \qquad \forall k \in K$

2. Cada posición contiene una única caja → Para cada posición (*i*, *j*), exactamente una caja la ocupa: $\sum_{k \in K} x_{kij} = 1 \qquad \forall i \in F,\ \forall j \in C$

3. Orden de prioridades → Si $p_k < p_q$ significa que k tiene mayor prioridad que q, por tanto k siempre debe estar en una fila igual o más cercana al acceso que q:

$$\sum_{i \in F} \sum_{j \in C} ix_{kij} \geq \sum_{i \in F} \sum_{j \in C} ix_{qij} \qquad \forall k,\ q \in K : p_k < p_q$$

4. Variables binarias para los valores de x → Indica si la caja *k* está situada en la posición (i,j):

$$x_{kij} \in \{0, 1\}\ \forall k \in K,\ \forall i \in F,\ \forall j \in C$$

Función Objetivo

La función objetivo minimiza el coste medio de la disposición de las cajas. Para una caja de prioridad p_k situada en la fila *i* existen *i-1* posiciones detrás de ella en su columna. El coste de una caja es la suma de las prioridades de las cajas que tiene delante; cambiando el orden de la suma, esto equivale a que cada caja sume su prioridad al coste de cada una de las i-1 cajas que tiene detrás. Por tanto, el coste total es la suma de$(i-1)p_k$ para cada caja. Al dividir entre l^2 se obtiene el coste medio:

$$min\ Z = \frac{1}{n} \sum_{k \in K} \sum_{i \in F} \sum_{j \in C} (i - 1) p_k x_{kij}$$

Implementación del modelo:

Para la implementación del modelo en el fichero *parte-1.mod* se han definido los siguientes conjuntos:

- *K*: conjunto de cajas, con cardinalidad $n = l^2$.
- *F*: conjunto de filas, con cardinalidad *l*.
- *C*: conjunto de columnas, con cardinalidad *l*.

Asimismo, se han establecido los siguientes grupos de restricciones:

- Una posición por caja (UnaPosicionPorCaja): restricción 1.
- Una caja por posición (UnaCajaPorPosicion): restricción 2.
- Orden de prioridades (OrdenPrioridades): restricción 3. Sólo se genera para las parejas de cajas con p[k] < p[q].
- Binariedad de las variables: $x_{kij} \in \{0,\ 1\}\ \forall k \in K,\ \forall i \in F,\ \forall j \in C$.

Finalmente, para verificar el correcto funcionamiento del modelo, ejecutamos varias pruebas:

- **CASO 1:** Solución única (sin margen de optimización)



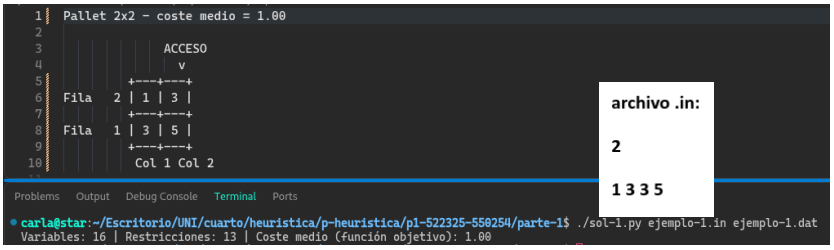
Todas las prioridades son distintas, de modo que la restricción de prioridades decide por sí sola qué cajas van en cada fila: {1, 2, 3} en la fila 3, {4, 5, 6} en la fila 2 y {7, 8, 9} en la fila 1.

- **CASO 2:** Todas las prioridades iguales



Como no existe ninguna pareja con pk < pq, no se genera ninguna restricción de prioridad y sólo quedan las 18 de asignación.

- **CASO 3:** Prioridades repetidas entre dos filas



Las dos cajas de prioridad 3 no se restringen entre sí, por lo que el modelo puede elegir cuál de ellas va a la fila del acceso y cuál a la fila 1.

PARTE 2. MONTAJE VOLUMÉTRICO

Modelización del problema:

Tipo de problema

En esta segunda parte se amplía el problema anterior permitiendo disponer las cajas en varios niveles sobre el pallet. Además de las prioridades, se deben considerar el peso y la capacidad de carga de las cajas para garantizar que se cumplen las restricciones de apilamiento.

Este apartado sigue siendo un problema de **asignación**: las $n = h \cdot l^2$ cajas se asignan a las $h \cdot l^2$ posiciones (i, j, z) del pallet, de modo que cada caja ocupa una posición y cada posición recibe una caja. A diferencia de la Parte 1, el problema **puede ser infactible**. Si las capacidades no permiten apilar los h niveles respetando los pesos (por ejemplo, con $h = 2$ y todas las capacidades iguales a 0), no existe ninguna disposición válida y GLPK indica que el problema no tiene solución factible. Siempre es **acotado**.

Parámetros

Se conservan los parámetros utilizados en la Parte 1, modificando el valor de n y añadiendo:

$n = h*l^2$
h: número de niveles.
w_k: peso de la caja k.
v_k: capacidad de la caja k.
N = {1,....,h}: conjunto de niveles.

Variables de decisión

$x_{kijz} \in \{0,\ 1\} \rightarrow$ 1 si la caja *k* se coloca en la fila *i*, la columna *j* y el nivel *z*; 0 en otro caso.

Se define una variable $\forall k \in K,\ \forall i \in F,\ \forall j \in C,\ \forall z \in N$

Restricciones

1. Cada caja ocupa una única posición: $\sum_{i \in F} \sum_{j \in C} \sum_{z \in N} x_{kijz} = 1 \qquad \forall k \in K$
2. Cada posición contiene una única caja: $\sum_{k \in K} x_{kijz} = 1 \qquad \forall i \in F,\ \forall j \in C,\ \forall z \in N$
3. Prioridad únicamente dentro del mismo nivel → si la caja q, de menor prioridad, está en la fila i del nivel z, entonces k no puede estar en ninguna fila por detrás de i de ese mismo nivel ($i' < i$) . Si k y q están en niveles distintos, la restricción no les afecta. Para i = 1 no hay filas por detrás, así que se omite:

$$\sum_{j \in C} x_{qijz} + \sum_{i' \in F : i' < i} \sum_{j \in C} x_{ki'jz} \leq 1 \qquad \forall z \in N,\ \forall i \in F : i > 1,\ \forall k, q \in K : p_k < p_q$$

4. Capacidad de carga → El lado izquierdo es el peso total de las cajas apiladas encima de la posición (*i*, *j*, *z*), es decir, en la misma fila y columna y en niveles superiores. El lado derecho es la capacidad de la caja que ocupa esa posición: por la restricción 2 hay exactamente una, así que la suma sólo tiene un término no nulo. En el nivel superior ($z = h$) no hay nada encima, por lo que se omite: $\sum_{k \in K} \sum_{z' \in N : z' > z} w_k x_{kijz'} \leq \sum_{k \in K} \gamma_k x_{kijz} \qquad \forall i \in F,\ \forall j \in C,\ \forall z \in N : z < h$
5. Variables binarias para los valores de x → Indica si la caja *k* está situada en la posición (i,j,z):

$$x_{kijz} \in \{0,\ 1\} \qquad \forall k \in K,\ \forall i \in F,\ \forall j \in C,\ \forall z \in N$$

Función Objetivo

La función objetivo minimiza el coste medio de la disposición de las cajas. El coste no depende del nivel, por lo que se calcula teniendo en cuenta únicamente la fila y la columna. Para una caja de prioridad (p_k) situada en la fila (i), existen (i-1) cajas detrás de ella en su columna, por lo que su aportación al coste es $(i - 1)\, p_k$. Al sumar las aportaciones de todas las cajas y dividir entre n, se obtiene el coste medio: $min\ Z = \frac{1}{n} \sum_{k \in K} \sum_{i \in F} \sum_{j \in C} \sum_{z \in N} (i - 1) \cdot p_k x_{kijz}$

Implementación del modelo:

Para la implementación del modelo en el fichero *parte-2.mod* se han definido los siguientes conjuntos:

- *K*: conjunto de cajas, con cardinalidad $n = h \cdot l^2$.
- *F*: conjunto de filas, con cardinalidad *l*.
- *C*: conjunto de columnas, con cardinalidad *l*.
- *N*: conjunto de niveles, con cardinalidad *h*.

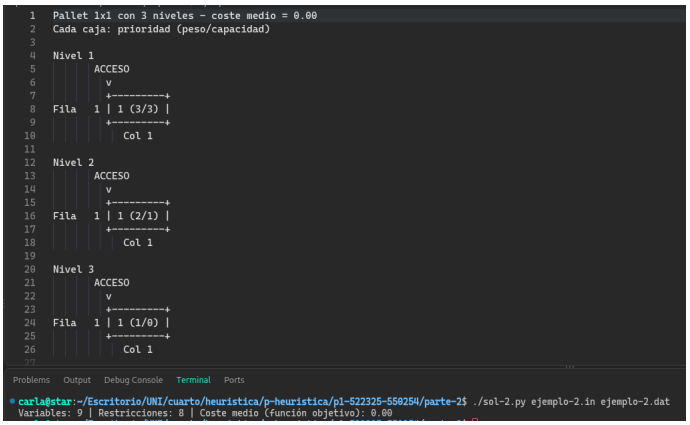
Asimismo, se han establecido los siguientes grupos de restricciones:

- Una posición por caja (UnaPosicionPorCaja): restricción 1.

– Una caja por posición (UnaCajaPorPosicion): restricción 2.
– Prioridad dentro de cada nivel (OrdenPrioridades): restricción 3. Sólo se genera para las filas $i > 1$ y las parejas con p[k] < p[q].
– Capacidad de carga (Capacidad): restricción 4. Sólo se genera para los niveles $z < h$.
– Binariedad de las variables: $x_{kijz} \in \{0,\ 1\}\ \forall k \in K,\ \forall i \in F,\ \forall j \in C,\ \forall z \in N$.

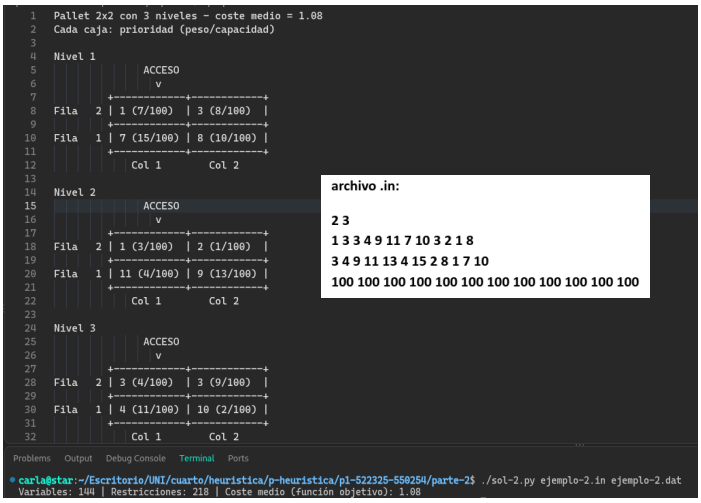
Finalmente, para verificar el correcto funcionamiento del modelo, ejecutamos varias pruebas:

- **CASO 1:** Solución única (sin margen de optimización)



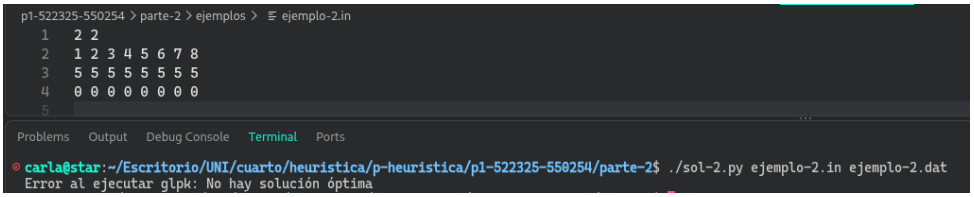
Un pallet de una sola posición y tres niveles, por lo que las tres cajas forman una única pila. La única forma de cumplir las capacidades es colocar abajo la caja de peso 3 (capacidad 3, soporta 2 + 1), en medio la de peso 2 (capacidad 1, soporta 1) y arriba la de peso 1. Cualquier otro orden supera alguna capacidad, así que la solución es única.

- **CASO 2:** Solución óptima (con margen de optimización)



Es el ejemplo del enunciado con todas las capacidades a 100, de modo que la capacidad deja de limitar. El óptimo coloca en las filas del acceso las seis cajas de mayor prioridad (1, 1, 2, 3, 3, 3) y obtiene un coste medio de 13/12 = 1,08. Con las capacidades reales, el modelo tiene que elegir entre las disposiciones válidas la de menor coste y acaba sustituyendo una caja de prioridad 3 por una de prioridad 4 (coste 1,17).

- **CASO 3:** Problema infactible



Ninguna caja puede soportar peso, pero con $h = 2$ todas las cajas del nivel 1 tienen otra encima. No existe ninguna disposición válida: GLPK responde PROBLEM HAS NO PRIMAL FEASIBLE SOLUTION y el script muestra un mensaje de error sin escribir ninguna solución.

PARTE 3. OPTIMIZACIÓN DE PALLETS INCOMPLETOS

Modelización del problema:

Tipo de problema

En esta parte se permite que el pallet contenga posiciones vacías.

Se trata de un problema **condicional**, y no de un problema de asignación como en las partes anteriores. Al haber posiciones vacías, el coste de cada posición deja de ser una expresión fija y pasa a depender de una condición: si la posición está vacía su coste es 0 y, si está ocupada, es la suma de las prioridades de las cajas que tiene delante. Esta condición se modela con la variable binaria y_{ijz} y una constante M_i. **Puede ser infactible** y siempre es **acotada**.

Parámetros

Para este apartado, los parámetros a tener en cuenta son los mismos que en la Parte 2. Sin embargo, ahora el pallet puede no estar completo: $1 \leq n \leq hl^2$. Además, se añade el parámetro $p_{max} = max_{k \in K} p_k$: el mayor valor numérico de prioridad, que se usa para fijar la constante $M_i = (l - i) \cdot p_{max}$ de la restricción condicional del coste (en GLPK no se puede usar infinito). Es la cota más ajustada posible: delante de la fila i hay como mucho $l - i$ cajas, todas con prioridad ≤ p_{max} .

Variables de decisión

$x_{kijz} \in \{0, 1\} \rightarrow$ 1 si la caja k se coloca en la fila i, columna j y nivel z; 0 en otro caso.

$y_{ijz} \in \{0, 1\} \rightarrow$ 1 si la posición de la fila i, columna j y nivel z está vacía; 0 en otro caso.

$u_{ijz} \geq 0 \rightarrow$ coste de la posición de la fila i, columna j y nivel z: suma de las prioridades de las cajas situadas delante de ella en su misma columna y nivel. Sólo se define para $i < l$, porque la fila del acceso no tiene nada delante y su coste siempre es 0.

Restricciones

1. Cada caja ocupa una única posición: $\sum_{i \in F} \sum_{j \in C} \sum_{z \in N} x_{kijz} = 1 \quad \forall k \in K$
2. Cada posición está ocupada por una caja o está vacía: $\sum_{k \in K} x_{kijz} + y_{ijz} = 1 \quad \forall\, i \in F,\ j \in C,\ z \in N$

   Como el número de cajas puede ser menor que el número máximo de posiciones, algunas variables y tomarán el valor 1.
3. Orden de las prioridades → Si $p_k < p_q$ entonces k tiene mayor prioridad que q, por lo que k debe estar en una fila igual o más cercana al punto de acceso que q:

   $$\sum_{j \in C} x_{qijz} + \sum_{i' \in F} \sum_{i' < i} \sum_{j \in C} x_{ki'jz} \leq 1 \qquad \forall z \in N,\ \forall i \in F : i > 1,\ \forall k, q \in K : p_k < p_q$$

   Esta restricción evita que una caja de mayor prioridad se encuentre detrás de una caja de menor prioridad dentro del mismo nivel.

4. Las posiciones vacías tienen la prioridad mínima → Una posición vacía tiene prioridad infinita así que debe comportarse como una posición situada detrás de cualquier caja:

$$y_{ijz} + \sum_{i'\epsilon F}\sum_{i'<i\, j'\epsilon C} x_{ki'j'z} \leq 1 \quad \forall z \in N,\ \forall i \in F:\ i\ >\ 1,\ \forall j \in C,\ \forall k \in K$$

5. Restricción de capacidad, una caja debe soportar el peso de todas las cajas situadas encima de ella en su misma fila y columna. Para cada posición que no pertenece al nivel superior:

$$\sum_{k\epsilon K}\sum_{z'\epsilon N,\, z'>z} w_k x_{kijz'} \leq \sum_{k\epsilon K} \gamma_k x_{kijz} \quad \forall\, j\epsilon C,\ i\epsilon F,\ z\epsilon N : z < h$$

6. No hay cajas sobre posiciones vacías: $y_{ijz} \leq y_{ij(z+1)}\ \ \forall i \in F,\ \forall j \in C,\ \forall z \in N:\ z\ <\ h$

   Si una posición está vacía, la de encima también lo está. La restricción de capacidad no basta para garantizarlo, porque una caja de peso 0 cumpliría $0 \leq 0$ aunque estuviera sobre un hueco. Sobre una posición vacía sí puede haber otras posiciones vacías.

7. Coste de cada posición **(restricción condicional)**:

   El coste de una posición depende de si está vacía o no. Es una condición del tipo «si a = 1 entonces b = c; si no, b = d», que se resuelve con una constante M muy grande:

$$si\ a\ =\ 1 \Longrightarrow b\ =\ c\text{:}\quad c\ -\ M{\cdot}(1\ -\ a) \leq b \leq c\ +\ M{\cdot}(1\ -\ a)$$
$$si\ a\ =\ 0 \Longrightarrow b\ =\ d\text{:}\quad d\ -\ M{\cdot}a \leq b \leq d\ +\ M{\cdot}a$$

   Cuando a = 1, el primer par de desigualdades fuerza b = c, mientras que el segundo queda desactivado porque M es muy grande.

   Cuando a = 0 ocurre al revés: el segundo par fuerza b = d y el primero queda desactivado. En este caso, a = 1 − y_{ijz} (la posición está ocupada), b = u_{ijz}, c es la suma de las prioridades de las cajas de delante y d = 0: $S_{ijz} = \sum_{k\in K}\sum_{i'\in F,\, i'>i} p_k x_{ki'jz}$

   La restricción se divide en tres grupos:

   * 7a (coste mínimo): si la posición está ocupada (y_{ijz} = 0), su coste no puede ser menor que la suma de las prioridades de delante. Si está vacía, se resta M_i y la restricción queda desactivada: $u_{ijz} \geq S_{ijz} - M_i y_{ijz} \quad \forall i \in F:\ i < l,\ \forall j \in C,\ \forall z \in N$

   * 7b (coste máximo): si la posición está ocupada, su coste no puede ser mayor que esa misma suma. Junto con 7a, obliga a que u_{ijz} sea exactamente la suma. Si está vacía, se suma M_i y también queda desactivada: $u_{ijz} \leq S_{ijz} + M_i y_{ijz} \quad \forall i \in F:\ i < l,\ \forall j \in C,\ \forall z \in N$

   *7c (coste nulo): si la posición está vacía (y_{ijz} = 1), el lado derecho vale 0, así que u_{ijz} = 0. Si está ocupada, el límite es M_i : $u_{ijz} \leq M_i(1 - y_{ijz}) \quad \forall i \in F:\ i < l,\ \forall j \in C,\ \forall z \in N$

<u>Función Objetivo</u>

En un primer momento pensamos en utilizar la misma función objetivo que en la Parte 2, ya que el coste de cada caja depende únicamente de su prioridad y de la fila en la que se encuentra, independientemente del nivel que ocupe. Sin embargo, al probar ejemplos nos dimos cuenta de que no resolvía el problema de forma óptima. El término $(i\ -\ 1){\cdot}p_k$ cobra la prioridad de la caja k una vez

por cada una de las $i - 1$ posiciones que tiene detrás, suponiendo que todas están ocupadas. Si alguna está vacía, su coste es 0, pero la fórmula sigue sumando la prioridad de la caja por ella, es decir, suma prioridades aunque detrás sólo haya huecos.

Por eso, la nueva función objetivo calcula el coste por posiciones, sumando las variables u_{ijz}, cuyo valor fija la restricción condicional 7: $min\ Z = \frac{1}{n} \sum_{i \in F : i < l} \sum_{j \in C} \sum_{z \in N} u_{ijz}$

Implementación del modelo:

Para la implementación del modelo en el fichero parte-3.mod se han definido los mismos conjuntos que en la Parte 2, teniendo en cuenta que:

- *K*: conjunto de cajas, con cardinalidad n, donde $1 \leq n \leq h \cdot l^2$.

Ahora, n ya no se calcula como $h \cdot l^2$, sino que se lee del fichero de datos. Además de las variables x_{kijz}, se definen las variables binarias y_{ijz}, que indican si una posición está vacía, y las variables continuas $u_{ijz} \geq 0$ (u), que representan el coste de cada posición. Las variables u_{ijz} sólo se crean para las filas $i < l$, porque la fila del acceso no tiene ninguna caja delante y su coste siempre es 0.

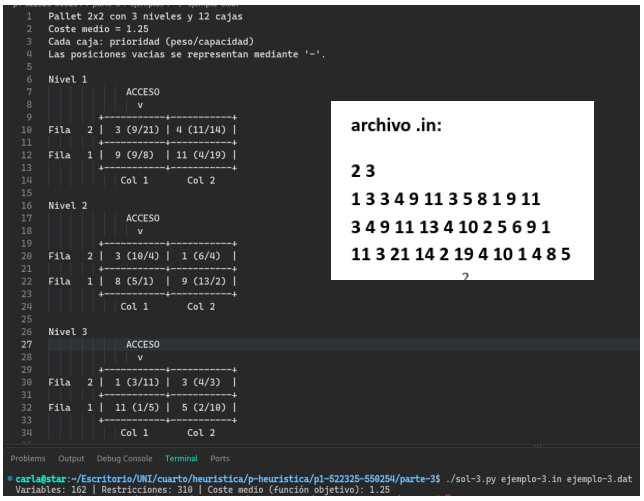
Asimismo, se han establecido los siguientes grupos de restricciones:

- Una posición por caja (UnaPosicionPorCaja): restricción 1.
- Cada posición tiene una caja o está vacía (UnaCajaOVacia): restricción 2. Sustituye a la restricción de una caja por posición de la Parte 2.
- Prioridad dentro de cada nivel (OrdenPrioridades): restricción 3. Sólo se genera para las filas $i > 1$ y las parejas con $p_k < p_q$.
- Las posiciones vacías tienen la prioridad mínima (OrdenVacios): restricción 4. Sólo se genera para las filas $i > 1$.
- Capacidad de carga (Capacidad): restricción 5. Sólo se genera para los niveles $z < h$.
- No hay cajas sobre posiciones vacías (NoSobreVacio): restricción 6. Sólo se genera para los niveles $z < h$.
- Coste de cada posición: restricción 7, dividida en CostePosicion (7a), CosteOcupada (7b) y CosteVacia (7c). Sólo se genera para las filas $i < l$.

El parámetro p_{max} se calcula dentro del modelo y se usa para la constante $M_i = (l - i) \cdot p_{max}$.

Finalmente, para verificar el correcto funcionamiento del modelo, ejecutamos varias pruebas:

- CASO 1: Pallet completo (equivalente a la Parte 2)



Con $n = h \cdot l^2$ no queda ninguna posición vacía y se obtiene el mismo coste medio que en la Parte 2. La única diferencia en este caso es el número de restricciones utilizadas: 162 variables y 310 restricciones, frente a 144 y 212 en la Parte 2.

CASO 2: Pallet incompleto



En este caso hay n = 9 cajas para un pallet de 2×2 con 3 niveles, es decir, 12 posiciones. Quedan por tanto tres posiciones vacías, y el modelo tiene que decidir dónde ponerlas además de colocar las cajas. Los huecos van al fondo del pallet (fila 1) para que no tapen el acceso, y nunca debajo de una caja. El coste más bajo posible se consigue poniendo delante de esas tres cajas las de mayor prioridad (1, 3 y 3).

CASO 3: Problema infactible:



Dos cajas en un pallet de una sola posición con dos niveles. Una tiene que ir encima de la otra, pero ninguna soporta peso. No existe ninguna disposición válida: GLPK responde PROBLEM HAS NO PRIMAL FEASIBLE SOLUTION y el script muestra un mensaje de error sin escribir ninguna solución.

PARTE 4. OPTIMIZACIÓN DEL MONTAJE EN UNA CANTIDAD ARBITRARIA DE PALLETS INCOMPLETOS

Modelización del problema:

En la última parte se amplía el problema permitiendo utilizar una cantidad arbitraria de pallets. Se busca determinar la distribución de las cajas y el número de pallets utilizados, minimizando el coste total, que incluye tanto el coste de disposición de las cajas como el coste fijo de preparar cada pallet.

Tipo de problema

Es de nuevo un problema **condicional**, ya que el coste depende de decisiones del propio modelo. Una posición sólo tiene coste si está ocupada, y solo se paga P por un pallet si contiene alguna caja. A diferencia de la Parte 3, siempre es **factible,** porque colocar cada caja sola en su propio pallet cumple todas las restricciones, y siempre es **acotado**, porque todos los costes son no negativos.

Parámetros

Se conservan los parámetros y conjuntos de la Parte 3: l, h, n, p_k, w_k, γ_k, $K = \{1, \ldots, n\}$, $F = C = \{1, \ldots, l\}$, $N = \{1, \ldots, h\}$, $A = \{1, \ldots, n\}$

Y se añade un nuevo parámetro P: coste fijo de montar un pallet.

Variables de decisión

$x_{kijza} \in \{0,1\}$: vale 1 si la caja k ocupa la posición (i,j,z) del pallet a.

$y_{ijza} \in \{0,1\}$: vale 1 si esa posición del pallet a está vacía.

$u_{ijza} \geq 0$: coste de la posición, como en la Parte 3; sólo se define para $i < l$.

$v_a \in \{0,1\}$: vale 1 si se utiliza el pallet a.

Restricciones

1. Cada caja ocupa una única posición: $\sum_{a\epsilon A}\sum_{i\epsilon F}\sum_{j\epsilon C}\sum_{z\epsilon N} x_{kijza} = 1 \quad \forall k \in K$

2. Cada posición contiene una caja o está vacía: $\sum_{k\epsilon K} x_{kijza} + y_{ijza} = v_a \quad \forall\, i\epsilon F,\ j\epsilon C,\ z\epsilon N,\ a \in A$

3. Cada pallet utilizado contiene alguna caja: $v_a \leq \sum_{k\epsilon K}\sum_{i\epsilon F}\sum_{j\epsilon C}\sum_{z\epsilon N} x_{kijza} \qquad \forall a \in A$

4. Orden de prioridades → Para las cajas k, q con $p_k < p_q$, la caja k debe estar en una fila igual o más cercana al acceso que q, dentro del mismo nivel y pallet:

$$\sum_{j\epsilon C} x_{qijza} + \sum_{i'\epsilon F\ i'<i}\sum_{j\epsilon C} x_{ki'jza} \leq 1 \qquad \forall a \in A,\ \forall z \in N,\ \forall i \in F:\ i > 1,\ k,\ q \in K:\ p_k < p_q$$

5. Las posiciones vacías tienen prioridad mínima, una posición vacía no puede tener detrás una caja: $y_{ijza} + \sum_{i'\epsilon F\ i'<i}\sum_{j'\epsilon C} x_{ki'j'za} \leq 1 \quad \forall a \in A,\ z \in N,\ \forall i \in F:\ i > 1,\ \forall j \in C,\ \forall k \in K$

6. Restricción de capacidad, cada caja debe soportar el peso total de las cajas situadas encima:

$$\sum_{k\epsilon K}\sum_{z'\epsilon N,\ z'>z} w_k x_{kijz'a} \leq \sum_{k\epsilon K} \gamma_k x_{kijza} \quad \forall\, a\epsilon A,\ j\epsilon C,\ i\epsilon F, z\epsilon N:\ z < h.$$

7. No hay cajas sobre posiciones vacías: $y_{ijza} \leq y_{ij(z+1)a} \quad \forall a \in A,\ i \in F,\ \forall j \in C,\ \forall z \in N:\ z < h$

8. Coste de cada posición. **(restricción condicional)** Se utiliza la misma formulación que en la parte 3, pero incluyendo el índice del pallet $S_{ijza} = \sum_{k\epsilon K}\sum_{i'\epsilon F, i'>i} p_k x_{ki'jza}$

   * 8a: si la posición está ocupada ($y_{ijza} = 0$), su coste no puede ser menor que la suma de las prioridades de las cajas que tiene delante. Si está vacía, se resta M_i y la desigualdad queda desactivada: $u_{ijza} \geq S_{ijza} - M_{iyijza} \qquad \forall a \in A,\ i \in F:\ i < l,\ j \in C,\ z \in N$

   * 8b: si la posición está ocupada, su coste no puede ser mayor que esa misma suma. Junto con la primera, obliga a que $u_{ijza} = S_{ijza}$. Si está vacía, se suma M_i y también queda desactivada: $u_{ijza} \leq S_{ijza} + M_{iyijza} \qquad \forall a \in A,\ i \in F:\ i < l,\ j \in C,\ z \in N$

   * 8c: si la posición está vacía ($y_{ijza} = 1$), el lado derecho vale 0, así que $u_{ijza} = 0$. Si está ocupada, el límite es M_i: $u_{ijza} \leq M_i(1 - y_{ijza}) \qquad \forall a \in A,\ i \in F:\ i < l,\ j \in C,\ z \in N$

Función Objetivo

Se minimiza la suma de los costes de disposición de todas las posiciones ocupadas y el coste fijo de cada pallet utilizado: $min\, Z = \sum_{a \epsilon A} (\sum_{i \epsilon F, i<l} \sum_{j \epsilon C} \sum_{z \epsilon N} u_{ijza} + Pv_a)$

Implementación del modelo:

Para la implementación del modelo en el fichero parte-4.mod se han definido los mismos conjuntos que en la Parte 3, teniendo en cuenta que:

– *K*: conjunto de cajas, con cardinalidad n. Ya no está limitada por el tamaño de un pallet, porque las cajas se pueden repartir entre varios.

– *A*: conjunto de pallets disponibles, con cardinalidad n. En el peor caso cada caja va en su propio pallet, así que nunca hacen falta más de n.

El coste de montar un pallet se lee de la primera línea del fichero de entrada y se guarda en el parámetro P.

Asimismo, se han establecido los siguientes grupos de restricciones:

– Una posición por caja (UnaPosicionPorCaja): restricción 1. Ahora la suma recorre también todos los pallets.

– Cada posición tiene una caja o está vacía (UnaCajaOVacia): restricción 2. El lado derecho es v[a] en lugar de 1.

– Un pallet utilizado contiene alguna caja (PalletUsado): restricción 3.

– Prioridad dentro de cada nivel y pallet (OrdenPrioridades): restricción 4. Sólo se genera para las filas $i > 1$ y las parejas con $p_k < p_q$.

– Las posiciones vacías tienen la prioridad mínima (OrdenVacios): restricción 5. Sólo se genera para las filas $i > 1$.

– Capacidad de carga (Capacidad): restricción 6. Sólo se genera para los niveles $z < h$.

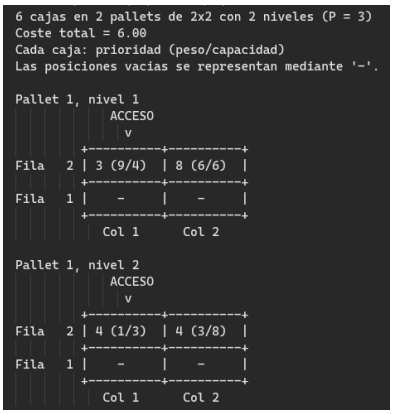
– No hay cajas sobre posiciones vacías (NoSobreVacio): restricción 7. Sólo se genera para los niveles $z < h$.

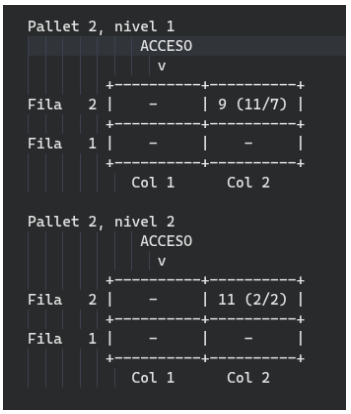
– Coste de cada posición: restricción 8, dividida en CostePosicion (8a), CosteOcupada (8b) y CosteVacia (8c). Sólo se genera para las filas $i < l$.

Igual que en la Parte 3, el parámetro p_{max} se calcula dentro del modelo y se usa para la constante $M_i = (l - i) \cdot p_{max}$ de la restricción 8. La función objetivo (CosteTotal) suma las variables u de todos los pallets y P por cada pallet utilizado. El script sol-4.py escribe la disposición de cada nivel de cada pallet utilizado y muestra por pantalla el número de variables, el de restricciones y el coste total; el número de pallets utilizados se escribe en el fichero de solución.

Finalmente, para verificar el correcto funcionamiento del modelo, ejecutamos varios casos de prueba:

- **CASO 1**: Problema con P=3:



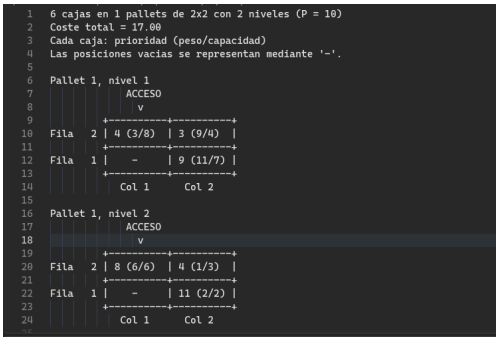


Con P = 3 compensa montar un segundo pallet, porque así ninguna caja tiene otra delante y el coste de disposición es 0.

archivo .in:

2 2 3
4 8 11 3 4 9
3 6 2 9 1 11
8 6 2 4 3 7

- **CASO 2**: Mismo ejemplo con P = 10.



Un segundo pallet ya cuesta más de lo que ahorra. Todas las cajas caben en uno con un coste de disposición de 7 (la 9 queda detrás de la 3 y la 11 detrás de la 4), así que el total es 10 + 7 = 17.

ANÁLISIS DE RESULTADOS

Para analizar cada parte hemos generado instancias aleatorias de distintos tamaños: prioridades entre 1 y 10, pesos entre 1 y 5 y capacidades entre 10 y 20. Cada tiempo incluye la generación del .dat, la ejecución de GLPK y la escritura de la solución. Ninguna parte puede ser no acotada: todas las variables están acotadas (x, y y v son binarias y $0 \leq u \leq Mi$) y el objetivo es una suma de términos no negativos. Las diferencias entre partes están en la factibilidad, en el número de soluciones óptimas y, sobre todo, en lo que le cuesta a GLPK demostrar la optimalidad.

Complejidad, factibilidad y número de óptimos

Parte	¿Se puede resolver sin PL?	¿Puede ser infactible?	¿Más de un óptimo?
1	Sí: ordenar por prioridad desde el acceso	No	Sí: permutar cajas de una fila
2	No: las capacidades acoplan los niveles	Sí (CASO 3)	Sí: intercambiar pilas de una fila
3	No: además hay que decidir los huecos	Sí (CASO 3)	Sí: como en la Parte 2
4	No: generaliza el bin packing (NP-difícil)	No: una caja por pallet	Sí: permutar los pallets

Parte 1:

La Parte 1 se puede resolver sin Programación Lineal. Como todas las posiciones están ocupadas y la restricción de prioridades obliga a que una caja nunca tenga detrás otra de mayor prioridad, basta con ordenar las cajas por prioridad (O(n log n)) y llenar las filas empezando por la del acceso.

Tamaño y tiempo de resolución

l	n = l²	Variables	Restricciones	Tiempo (s)
3	9	81	50	0,02
4	16	256	141	0,02
6	36	1.296	646	0,07
8	64	4.096	1.926	0,35
10	100	10.000	4.672	1,90
12	144	20.736	9.584	11,71
15	225	50.625	23.100	74,90

GLPK lo resuelve con facilidad: el número de variables crece como l^4, el de restricciones depende de los datos y el tiempo crece sólo por el tamaño del modelo.

Parte 2:

Al añadir niveles y capacidades ya no existe una regla sencilla que dé el óptimo: la prioridad pide colocar ciertas cajas en las filas del acceso, pero la capacidad puede impedir apilarlas como convendría. Por eso hace falta un modelo de optimización.

Tamaño y tiempo de resolución

l	h	n = h·l²	Variables	Restricciones	Tiempo (s)
2	3	12	144	200	0,02
3	3	27	729	1.944	0,06
5	2	50	2.500	8.933	0,47
6	2	72	5.184	23.110	2,79
4	4	64	4.096	21.644	4,90
3*	3*	27*	729	1.914	> 900

En este caso, las variables son $(h \cdot l^2)^2$ y las restricciones crecen aún más deprisa que en la Parte 1, porque las de prioridad se repiten en cada nivel y en cada fila. En las cinco primeras filas de la tabla las capacidades son muy superiores al peso que soportan, por lo que el problema sigue siendo fácil para GLPK. Aun así, el número de niveles influye: con l = 4 y h = 4 (64 cajas) se tarda más que con l = 6 y h = 2 (72 cajas), porque con más niveles las capacidades empiezan a limitar y GLPK tiene que explorar más nodos. En cambio, con capacidades ajustadas (fila marcada con *: capacidades del mismo orden que el peso que deben soportar), una instancia del mismo tamaño que la segunda fila no termina en 15 minutos. Por tanto, en esta parte el tamaño máximo depende más de los datos que de l y h.

Parte 3:

En este caso, además de colocar las cajas hay que decidir qué posiciones quedan vacías, y el coste de cada posición depende de esa decisión, por lo que necesitamos Programación Lineal.

Tamaño y tiempo de resolución

l	h	n	Variables	Restricciones	Tiempo (s)
2	2	6	60	86	0,03
2	3	9	126	208	0,62
3	2	6	138	206	0,04

l	h	n	Variables	Restricciones	Tiempo (s)
3	2	9	192	313	1,45
3	2	12	246	468	3,08
3	2	15	300	663	67,11
3	2	18	354	842	0,04
3	3	9	288	480	0,07
3	3	18	531	1.305	> 300
3	3	24	693	2.085	> 300
4	2	20	696	1.662	> 300

Las instancias que se resuelven son mucho más pequeñas que en la Parte 2, aunque para un mismo pallet el modelo sólo añade las variables y y u. El motivo es la restricción condicional: con la constante Mi la relajación lineal es débil y GLPK tiene que ramificar mucho. La serie con l = 3 y h = 2 muestra que el tiempo no depende tanto de n como del número de formas de colocar los huecos. Con 6 cajas no hace falta apilar nada y con 18 el pallet está lleno, así que no hay huecos que decidir: en los dos casos se resuelve en 0,04 s. Los casos intermedios son los más difíciles, y el tiempo crece con n hasta las 15 cajas. Con 3 niveles y 18 o más cajas, o con l = 4, GLPK no termina en 5 minutos.

Parte 4:

En este caso, hay muchas soluciones óptimas equivalentes, porque los pallets son intercambiables: cualquier permutación de los pallets usados da el mismo coste. Esta simetría es la principal causa del tiempo de resolución. Tres instancias aleatorias por tamaño (l = h = 2, P = 3; pallets y coste de la primera, tiempo mínimo–máximo):

n	Variables	Restricciones	Pallets	Coste	Tiempo (s)
4	180	224–232	1	3	0,13–0,15
6	366	504	2	6	42,86–80,91
8	616	880	–	–	> 180

Con este modelo el tiempo se dispara a partir de 6 cajas. Efecto de P sobre el ejemplo del caso 1:

P	0	1	3	5	10	20	100
Pallets	3	2	2	2	1	1	1
Coste total	0	2	6	10	17	27	107
Tiempo (s)	0,19	28,09	27,27	34,02	0,96	0,85	0,63

P cambia tanto la solución como la dificultad: con P = 0 (el número de pallets es arbitrario) o con P ≥ 10 (basta un pallet) GLPK tarda menos de 1 s; con P entre 1 y 5 compensan dos pallets, pero demostrarlo obliga a comparar muchos repartos equivalentes y tarda unos 30 s.

Conclusiones

Las Partes 1 y 2 (con capacidades holgadas) son problemas de asignación con muchas variables, pero su relajación lineal ya da el óptimo entero y GLPK las resuelve prácticamente sin ramificar: el límite lo marca el tamaño del modelo (l = 15 en la Parte 1, unas 75 cajas en la Parte 2). En las Partes 3 y 4, en cambio, las restricciones condicionales con Mi debilitan la relajación y obligan a explorar muchos nodos, de modo que el límite baja a unas 15 cajas en la Parte 3 y a unas 6 en la Parte 4. En todos los casos influyen más los datos que el tamaño: capacidades ajustadas, un número intermedio de huecos o un P cercano al punto de equilibrio hacen difícil una instancia pequeña.